

PROGRAMACIÓN AVANZADA

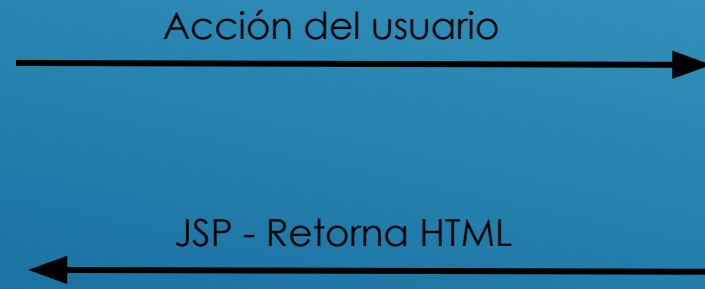
JSP
Java Server Page



Ing. Matias Macrino

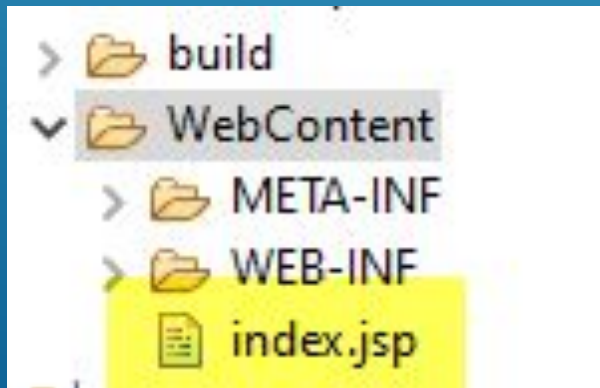
QUÉ ES JSP

- ▶ **Java Server Page:** Páginas de Servidor Java es una tecnología aplicada del lado del servidor, para la creación de páginas web dinámicas y/o aplicaciones web.
- ▶ Código java que se ejecuta en un servidor (server side).
- ▶ Retorna una nueva página HTML que se genera de manera dinámica, dependiendo de las acciones realizadas por el usuario (client side).



JAVA SERVER PAGE

- ▶ Se debe de almacenar el archivo con extensión *.jsp
- ▶ Estos archivos serán almacenados dentro del servidor web.
- ▶ Se trata de un archivo que contiene el formato HTML y dentro del mismo código Java puro.
- ▶ Todos los archivos jsp, se deben de alojar dentro del proyecto web al mismo nivel que la carpeta WEB-INF.



```
index.jsp
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Mi primer JSP</title>
6 </head>
7 <body>
8     <h1>Hola Mundo desde una JSP!!! =></h1>
9 </body>
10 </html>
```

JAVA SERVER PAGE

- ▶ **Tipo de código JSP:** todos los tipos de código JSP se expresa mediante el tag, al igual se realiza con HTML. Ya que las JSP son archivos HTML con código java embebido.
- ▶ **Expresiones:** `<%= linea de código java %>`
- ▶ **Scriptlets:** `<% varias líneas de código java %>`
- ▶ **Declarativas:** `<%! declaración de variables o métodos %>`

JAVA SERVER PAGE

- Expresiones: unica linea de código java.

```
index.jsp
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Mi primer JSP</title>
6 </head>
7 <body>
8     <h1>Hola Mundo desde una JSP!!! =)</h1>
9
10    <h3>Fecha y hora del sistema: <%= new java.util.Date() %></h3>
11 </body>
12 </html>
```



JAVA SERVER PAGE

► Scriptlet: varias lineas de código java.



```
*index.jsp
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Mi primer JSP</title>
6 </head>
7 <body>
8     <h1>Hola Mundo desde una JSP!!! =)</h1>
9     <%
10         for(int i = 1; i <= 5; i++){
11             out.println("Mostramos el mensaje N°: " + i + "<br>");
12         }
13     %>
14 </body>
15 </html>
```

JAVA SERVER PAGE

- Definiciones: declaraciones en código java

```
index.jsp
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Mi primer JSP</title>
6 </head>
7 <%!public int suma(int a, int b) {
8     return a + b;
9 }%>
10 <body>
11     <h1>Hola Mundo desde una JSP!!! =></h1>
12     La suma de 2 + 5 es: <%= suma(2, 5) %>
13 </body>
14 </html>
```



JAVA SERVER PAGE

- ▶ Definiciones: declaraciones en código java.
- ▶ No es una buena práctica tener mucho código java en mi archivo JSP.
- ▶ Por lo que estas definiciones pueden ser llevadas a clases java.
- ▶ Reemplazando esto por uso del tipo **Expresiones**
- ▶ Si no utilizando el import page.

```
package edu.usal;  
  
public class Calculos {  
    public static int suma(int a, int b) {  
        return a + b;  
    }  
}
```

```
<!DOCTYPE html>  
<%@page import="edu.usal.Calculos"%>  
<html>  
<head>  
    <meta charset="UTF-8">  
    <title>Mi primer JSP</title>  
</head>  
<body>  
    <h1>Hola Mundo desde una JSP!!! =)</h1>  
    La suma de 2 + 5 es: <%= Calculos.suma(2, 5) %>  
</body>  
</html>
```

```
<!DOCTYPE html>  
<html>  
<head>  
    <meta charset="UTF-8">  
    <title>Mi primer JSP</title>  
</head>  
<body>  
    <h1>Hola Mundo desde una JSP!!! =)</h1>  
    La suma de 2 + 5 es: <%= edu.usal.Calculos.suma(2, 5) %>  
</body>
```

JAVA SERVER PAGE

- ▶ Variables predefinidas:
- ▶ JSP cuenta con objetos implícitos, para utilizar en código java.
 - ▶ request: corresponde al objeto HttpServletRequest.
 - ▶ response: corresponde al objeto HttpServletResponse.
 - ▶ out: corresponde al objeto PrintWriter
 - ▶ session: corresponde al objeto HttpSession
 - ▶ config: corresponde al objeto ServletConfig

```
<%
    Enumeration<String> cabeceras = request.getHeaderNames();
    while(cabeceras.hasMoreElements()) {
        out.println("<br>" + cabeceras.nextElement());
    }
%>
```

JAVA SERVER PAGE

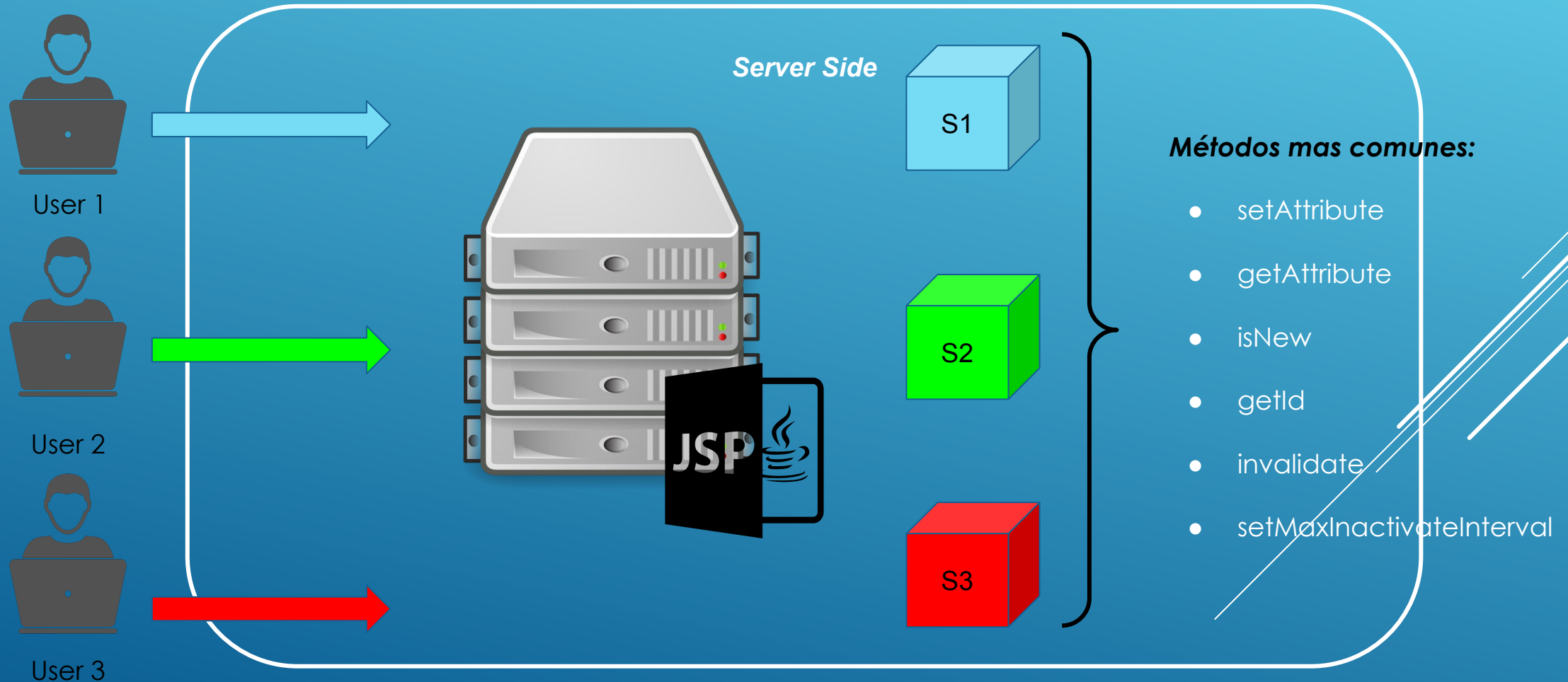
▶ Variables predefinidas:

- ▶ **Session:** ya que no contamos con manera de almacenar estados o almacenar acciones del usuario, en diversas tecnologías aplicadas a la web contamos con sesiones. Estas mismas son creadas para recordar y seguir las acciones de cierto usuario en un sitio web.
- ▶ Ejemplos de utilización:
 - ▶ Carritos web
 - ▶ Home Banking
 - ▶ Exámenes online.

JAVA SERVER PAGE

► Variables predefinidas:

► *Session:*



JAVA SERVER PAGE

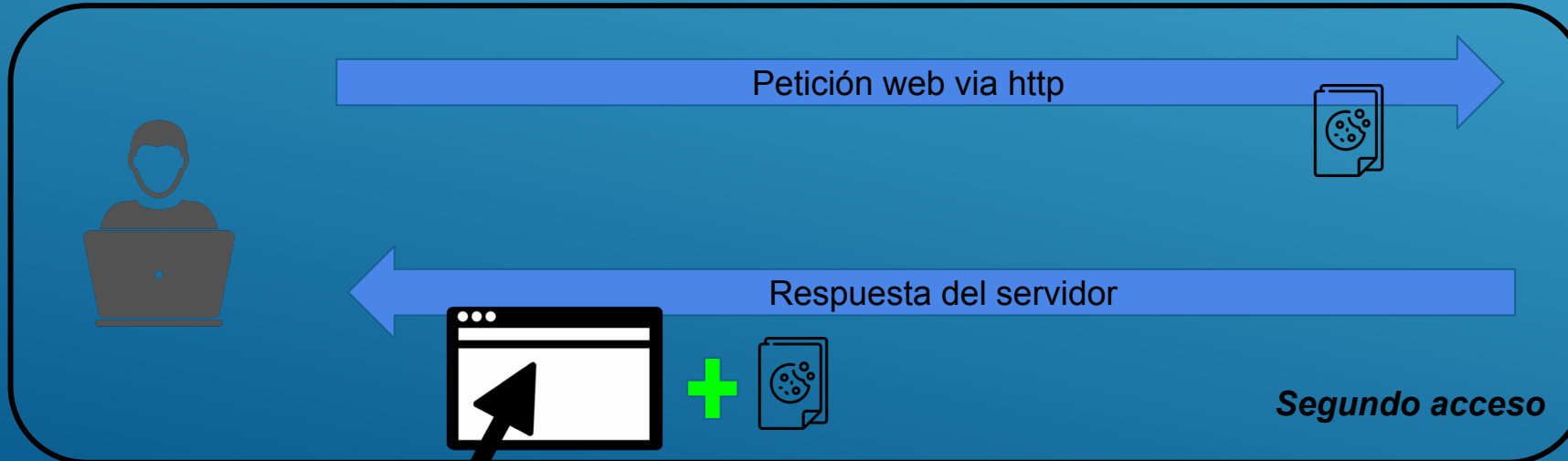
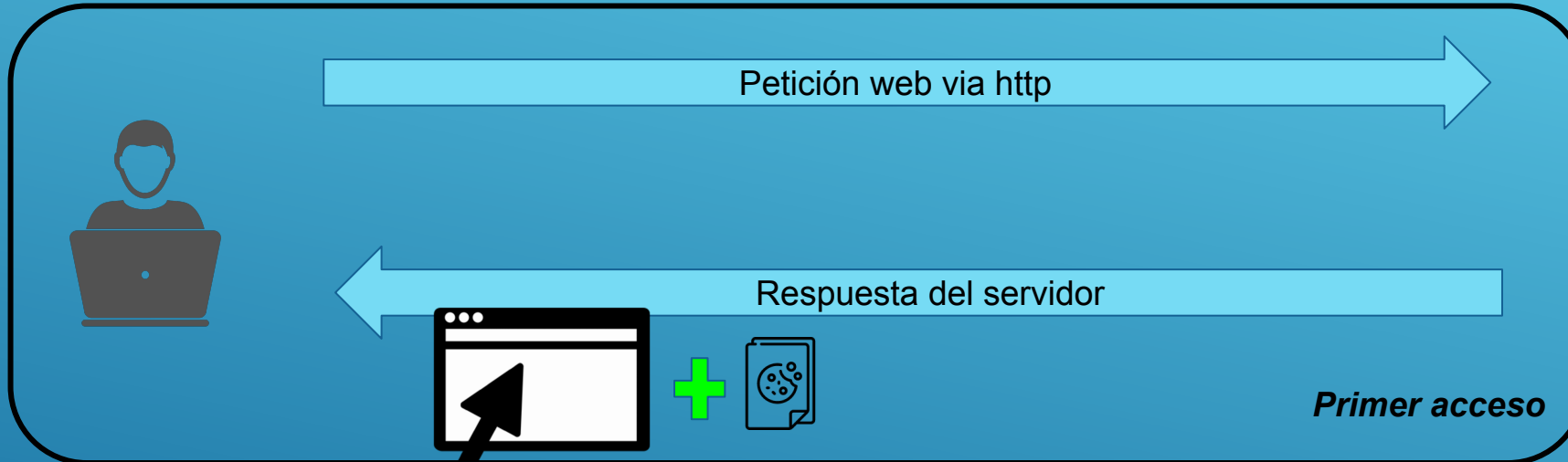
▶ Variables predefinidas:

- ▶ **Cookies:** pequeñas secciones de información textual que envía el servidor al browser. Son archivos de texto del formato clave - valor, que se almacenan en el browser.
- ▶ Ejemplos de utilización:
 - ▶ Presentar el sitio como se lo dejó anteriormente.
 - ▶ Poder guardar el visitante, sin necesidad de volver hacer el login

JAVA SERVER PAGE

► Variables predefinidas:

► Cookies:



JAVA SERVER PAGE

▶ JSP Tags:

- ▶ Librería de etiquetas, similares a las de HTML utilizadas solo para paginas JSP.
- ▶ Se utilizan para hacer más simple, el desarrollo, lectura y modificación de nuestras JSP.
- ▶ Nos permiten minimizar el código en la página web y que el mantenimiento sea más sencillo.
- ▶ ¿Como la utilizamos?: en nuestro proyecto debemos de importar las librerías necesarias y luego incorporarlas en nuestra JSP.

JAVA SERVER PAGE

▶ JSP Tags:

- ▶ JSTL es la librería estándar que se utilizara para los tags de JSP.
- ▶ Se compone de **Core tags, Formatting tags, SQL tags y XML tags.**
- ▶ Adicionalmente tiene un apartado de Funciones denominado **JSTL Functions.**
- ▶ Como contamos con JSTL, también podemos tener nuestras propias **custom tags**

JAVA SERVER PAGE

► Core Tags:

- **<c:if>**: es equivalente a la sentencia IF de cualquier lenguaje de programación.

Su sintaxis es la siguiente:

<c:if test="condición a evaluar"> código a ejecutar </c:if>

- **<c:forEach>**: equivalente a cualquier ciclo FOR de programación.

Su sintaxis es la siguiente:

<c:forEach var="var" items="items"> código a ejecutar </c:forEach>

JAVA SERVER PAGE

► Core Tags:

- **<c:choose>**: es equivalente a la sentencia SWITCH de cualquier lenguaje de programación.
Su sintaxis es la siguiente:

```
<c:choose>  
  <c:then>  
    Código a ejecutar  
  </c:then>  
  <c:then>  
    Código a ejecutar  
  </c:then>  
</c:choose>
```